

FuelSync — Complete Codebase Guide

> Hybrid athlete nutrition + training PWA. Tracks macros, cycles nutrition by training type, integrates with Strava, checks weather conditions, and uses AI (Gemini) for food estimation.

Table of Contents

- [Architecture Overview](#1-architecture-overview)
 - [Technology Stack](#2-technology-stack)
 - [File Map](#3-file-map)
 - [Data Storage — IndexedDB (Dexie)](#4-data-storage--indexeddb-dexie)
 - [Shared Types](#5-shared-types)
 - [Frontend Entry Points](#6-frontend-entry-points)
 - [Screens](#7-screens)
 - [Components](#8-components)
 - [Stores (Zustand)](#9-stores-zustand)
 - [Hooks](#10-hooks)
 - [API Layer](#11-api-layer)
 - [Utilities](#12-utilities)
 - [Business Logic — Nutrition Engine](#13-business-logic--nutrition-engine)
 - [Business Logic — Weather Service](#14-business-logic--weather-service)
 - [Security — PIN System](#15-security--pin-system)
 - [Cloudflare Worker — AI (Gemini)](#16-cloudflare-worker--ai-gemini)
 - [Cloudflare Worker — Strava OAuth](#17-cloudflare-worker--strava-oauth)
 - [Infrastructure — Docker + Nginx](#18-infrastructure--docker--nginx)
 - [Design System](#19-design-system)
 - [Key Business Rules](#20-key-business-rules)
-

1. Architecture Overview

Browser (PWA)

|

| — IndexedDB (Dexie) ← ALL data stored locally, no login required

| | — profile ← user body stats, Strava tokens

| | — food_logs ← every meal entry

| | — pin_state ← hashed PIN + lockout state

|

| — Cloudflare Worker: fuelsync-ai

| | — Gemini 2.5-flash-lite ← food estimation, meal suggestion, image analysis

|

| — Cloudflare Worker: fuelsync-strava

| | — Strava API ← OAuth, activity fetch, token refresh

Docker (home server @ danielmms.site)

| — foodaniel container ← Vite build → nginx static files

└─ (nginx proxies /api/ to foodaniel-backend:3000 – legacy, not used by PWA)

Key design principle: The app is **100% local-first**. No user account, no JWT, no backend calls for data. Everything lives in the browser's IndexedDB. The Cloudflare Workers are the only external dependencies — they proxy to Gemini (AI) and Strava (running data).

2. Technology Stack

LayerTechnologyWhy

|---|---|---|

UI Framework**React 18** + TypeScriptComponent model, hooks Build Tool**Vite**Fast HMR, ES module output PWA**VitePWA** (Workbox)Offline support, installable on phone Local DB**Dexie.js** (wrapper around IndexedDB)Typed, promise-based local database State**Zustand**Lightweight global state; persists weekly load to localStorage AI**Google Gemini 2.5-flash-lite**Free tier, fast, multimodal (image support) Edge Functions**Cloudflare Workers**Serverless, global, no cold start Worker Deploy**Wrangler v3** (CLI)Deploy workers from terminal StylingInline styles + CSS classes in `index.css`No CSS framework, full control Font**Inter** (Google Fonts)Clean, athletic look Running API**Strava API v3**OAuth activities + athlete stats Weather API**OpenWeatherMap**Current conditions by GPS coordinates Hosting**Docker + nginx** on home serverServes the static Vite build Tunnel**Cloudflare Tunnel**Exposes home server safely (no port forwarding)

3. File Map

FuelSync/

```
└─ Dockerfile                ← multi-stage: Node build → nginx

└─ nginx.conf                ← SPA routing + /api proxy

└─ CODEBASE.md               ← this file

|

└─ apps/

|   └─ web/                  ← the actual PWA (what users see)

|   |   └─ vite.config.ts    ← build config, PWA manifest, path
aliases

|   |   └─ src/

|   |   |   └─ main.tsx      ← React entry, mounts

|   |   |   └─ App.tsx       ← root shell: auth gate, tab nav,
Strava OAuth

|   |   |   └─ index.css     ← design system: CSS variables,
animations

|   |   |   └─ screens/

|   |   |   |   └─ AuthScreen.tsx    ← first-time profile setup

|   |   |   |   └─ PinScreen.tsx     ← PIN lock / unlock / setup

|   |   |   |   └─ HomeScreen.tsx    ← calories, macros, training,
runs, weather
```

| | | | — FoodScreen.tsx ← food logging (AI, photo, manual, suggest)

| | | | — ProfileSetupScreen.tsx ← edit body stats, display name, sign out

| | | | — components/

| | | | — TrainingPicker.tsx ← rest/strength/cardio/hybrid selector

| | | | — StravaCard.tsx ← Strava runs list + share card

| | | | — WeatherBanner.tsx ← conditions + training alert

| | | | — MacroBar.tsx ← horizontal macro progress bar

| | | | — MacroCard.tsx ← macro summary card

| | | | — RingProgress.tsx ← SVG circular progress ring

| | | | — store/

| | | | — authStore.ts ← user profile + PIN verified flag

| | | | — nutritionStore.ts ← today's log, targets, weekly load

| | | | — hooks/

| | | | — useNutrition.ts ← logDay, refreshWeather, getMacroBreakdown

| | | | — api/

| | | | | — client.ts ← workerPost/workerGet →
Cloudflare Workers

| | | | | — auth.ts ←
getProfile/createProfile/updateProfile (IndexedDB)

| | | | | — localFood.ts ← food CRUD + AI estimate
functions

| | | | | — strava.ts ← Strava
connect/disconnect/stats/refresh

| | | | | — lib/

| | | | | — db.ts ← Dexie database class + table
interfaces

| | | | | — pin.ts ← PBKDF2 PIN hashing, verify,
lockout, wipe

| | | | | — utils/

| | | | | — sounds.ts ← Web Audio API success sound
(C5→E5→G5)

| | | | | — runShareCard.ts ← Canvas-rendered run share
card (1080×1350)

| |

| — mobile/src/ ← shared business logic (used by web
too)

| — services/

```
| └─ nutritionEngine.ts ← TDEE, macro cycling, fatigue gate,
adherence score

| └─ weatherService.ts ← OpenWeatherMap fetch + danger alert
logic

|

└─ shared/types/

| └─ index.ts ← all TypeScript interfaces
(TrainingType, MacroTargets, etc.)

|

└─ workers/

    └─ ai/

        | └─ index.ts ← Cloudflare Worker: /estimate
        /describe /suggest /analyze

        | └─ wrangler.toml ← worker name, allowed origin, secret
        names

        └─ strava/

            └─ index.ts ← Cloudflare Worker: /auth-url
            /callback /refresh /stats

            └─ wrangler.toml ← worker name, APP_URL, WORKER_URL
```

4. Data Storage — IndexedDB (Dexie)

File: `apps/web/src/lib/db.ts`

The entire app stores data locally in the browser using IndexedDB (via Dexie). No server database.

Tables

profile — user body stats

```
interface LocalProfile {

  id?: number;

  displayName: string;

  weightKg: number;

  heightCm: number;

  age: number;

  gender: 'male' | 'female';

  activityLevel: 'sedentary' | 'light' | 'moderate' | 'very_active' |
  'extra_active';

  dailyGoal: number;

  // Strava OAuth tokens (stored here, not a separate table)

  stravaAccessToken?: string;
```



```
    stravaRefreshToken?: string;

    stravaExpiresAt?: number;    // Unix timestamp

    stravaAthleteName?: string;

    stravaAthletePic?: string;

}
```

Only ever has **one row**. `createProfile()` clears the table before adding.

`food_logs` — every meal logged

```
interface LocalFoodLog {

    id?: number;

    food_name: string;

    calories: number;

    protein: number;

    carbs: number;

    fat: number;

    weight_grams: number | null;

    meal_type: string;    // 'breakfast' | 'pre_workout' | 'lunch'
    | etc.

    image_url: string | null;
```

```

    logged_at: string;           // ISO datetime

    date: string;               // YYYY-MM-DD — indexed for fast date
                                queries
}

// Indexes: date, logged_at

```

pin_state — PIN lock

```

interface PinState {

    id?: number;

    hash: string;               // PBKDF2 hex hash

    salt: string;               // random 16-byte salt as hex

    attempts: number;           // wrong attempts in current lockout
                                window

    lockedUntil: number | null; // Date.now() timestamp

    totalAttempts: number;       // cumulative (resets on correct PIN)

    securityQuestion?: string;

    securityAnswerHash?: string;

    securityAnswerSalt?: string;

    securityAnswerAttempts?: number;
}

```

```
| }
```

Database Class

```
class FuelSyncDB extends Dexie {  
  
  profile: Table  
  
  food_logs: Table  
  
  pin_state: Table  
  
}  
  
export const db = new FuelSyncDB(); // singleton, used everywhere
```

5. Shared Types

File: `shared/types/index.ts`

These interfaces are the contract between the UI, nutrition engine, weather service, and Cloudflare Workers.

TypeDescription

|---|---|

TrainingType 'rest' \| 'strength' \| 'cardio' \| 'hybrid' — the 4 training modes
IntensityLevel 'low' \| 'moderate' \| 'high'
UserProfileBody stats: weightKg, heightCm, age, sex, activityLevel, goals
DailyLogOne day's training context: date, trainingType, plannedWorkoutTime, intensity

MacroTargets{ calories, proteinG, carbsG, fatG, carbTimingWindow? }

MacroBreakdownFull result from engine: TDEE + targets + rationale string

WeatherConditions{ tempC, humidity, uvIndex, description, city, timestamp }

EnvironmentAlert{ level, message, suggestedPivot?, pivotReason?,
extraHydrationMl? } WeeklyLoadRunning total: totalRunKm, totalStrengthSets,
legFatigueScore, recoveryScore, loggedRuns[] LoggedRun{ km, name, source:
'strava' \ | 'manual' } FoodEntryFull food entry with meal slot (used in snapshots)
DaySnapshotComplete day picture: log + targets + consumed + weather +
weeklyLoad

6. Frontend Entry Points

`apps/web/src/main.tsx`

Single line: mounts inside React StrictMode into `#root`.

`apps/web/src/App.tsx`

The root component. Controls the entire app lifecycle.

Startup flow:

- Shows loading spinner (animated dots)
- Calls `getProfile()` from IndexedDB
- If profile exists → checks if PIN is set (`hasPin()`)
- If PIN set → shows , otherwise marks pin verified
- If no profile → shows (first-time setup)
- Once authenticated + pin verified → shows main tabs

Strava OAuth handling:

After Strava redirects back to the app, the Worker puts tokens in URL query params (`?strava_access_token=...`). `App.tsx` detects these, calls `connectStrava()` to save them to IndexedDB, then clears the URL with `window.history.replaceState`.

Tab navigation:

Three tabs rendered in a fixed bottom nav:

- `home` →
- `food` →

- `profile` →

Each tab has an SVG icon. Active tab gets a blue top indicator bar with a glow shadow.

`apps/web/vite.config.ts`

- **Path aliases:** `@shared` → `../../shared`, `@mobile` → `../mobile/src` (so the web app can import the nutrition engine directly)
 - **VitePWA:** Generates a service worker (Workbox). `registerType: 'autoUpdate'` — SW updates automatically on new deploy. AI/Strava worker URLs are `NetworkOnly` — never cached, always fresh.
 - **PWA manifest:** App name "FuelSync", standalone display (no browser chrome), portrait orientation.
-

7. Screens

`AuthScreen.tsx` — First-time Profile Setup

Shown when no profile exists in IndexedDB. Collects:

- Display name
- Weight (kg), Height (cm), Age
- Gender (male/female)
- Activity level (sedentary → extra active)
- Daily calorie goal

On submit: calls `createProfile()` which writes to IndexedDB, then `setUser()` into `authStore`. Also prompts to set a PIN after profile creation.

`PinScreen.tsx` — PIN Lock

Shown before entering the app if a PIN has been set. Uses the numpad UI.

States:

- Normal entry → 4-digit PIN input
- Wrong PIN → shows attempts left + lockout countdown
- Locked → countdown timer shown
- Wiped → after 15 total wrong attempts, the entire database is deleted

Features:

- "Forgot PIN?" → security question flow (if set)
- Change PIN (accessible from ProfileSetupScreen)

HomeScreen.tsx — Dashboard

The main dashboard. Shows everything in one scrollable view.

Sections (top to bottom):

- **Header gradient band** — blue gradient with "Good Morning/Afternoon/Evening, NAME" + date chip
- **Profile alert** — orange warning if body stats are incomplete (can't compute macros without them)
- **Calorie ring** — SVG: shows consumed/target calories as a circular arc. Turns red if over target.
- **Macro pills row** — × 3: protein (green), carbs (orange), fat (purple). Each shows current/target with a small progress bar. Turns red if over.
- **Recovery banner** — computed from `totalRunKm` and number of runs this week. States: FRESH / ACTIVE / BUILDING / LOADED / OVERLOADED with matching color.
- **Training mode** — to select today's training type. Shows RESET button if already selected.
- **Workout time input** — that feeds back into macro computation via `logDay()`. Only shown for non-rest days.
- **Carb window** — shown if workout time is set. Displays the optimal carb

timing window (± 2 h around workout time) with target carb grams for that window.

- **Logged runs** — list of this week's runs (Strava + manual). Each run shows colored dot (orange=Strava, green=manual), name (tap to rename inline), distance, × to remove. "LOG +" button opens a form to add a manual run by name + km.
- **Strava card** — embedded here.
- **Weather conditions** — if weather has been fetched (on mount).

Key functions in HomeScreen:

FunctionWhat it does

|---|---|

`sumLogs(logs)`Adds up calories/protein/carbs/fat from today's food logs

`recoveryFromKm(km, runs)`Maps weekly km total to a recovery label + color + score

`calRingSVG` circle component. `strokeDasharray` animates the arc fill

`MacroPill`Macro progress card with colored top border `handleSelectType`Calls

`logDay()`, handles leg fatigue gate (confirm override dialog)

`FoodScreen.tsx` — Food Log

The most complex screen. Four modes in a bottom sheet.

Food list (main view):

- Entries grouped by meal type (Breakfast, Pre-Workout, Lunch, etc.)
- Each entry rendered as
- Calorie counter at top with macro breakdown grid
- Blue FAB (+) button to open the sheet

component:

Shows food name, P/C/F macros, calorie count. Three action buttons at bottom:

- **Edit** → populates manual form with this entry's data

- **Remove** → deletes from IndexedDB, triggers undo toast
- **Info ▼** → expands to show detailed macro grid + kcal/100g density

Undo Toast:

After deleting a food, a dark toast appears at the bottom for 5 seconds with an "Undo" button. Uses `useRef` for the timer so it can be cleared if user taps Undo. On undo, re-adds the entry to IndexedDB via `addLog()`.

Sheet modes (tabs: AI / Photo / Manual / Suggest):

AI Mode

- User types a free-text description: "2 eggs and a banana" or "bowl of oatmeal with berries"
- Calls `estimateByDescription()` → hits Cloudflare AI Worker `/describe`
- Gemini returns full macros + ingredients breakdown
- Transitions to Manual mode with fields pre-filled
- User reviews and taps "Log Fuel →"

Photo Mode

- — opens camera on mobile
- Converts image to base64, calls `analyzeByImage()` → hits AI Worker `/analyze`
- Same flow as AI mode: fills Manual form with result

Manual Mode

- Direct entry: food name, amount (grams or qty text), protein/carbs/fat
- **g/qty toggle**: "g" mode = number input → macros scale proportionally when weight changes. "qty" mode = text input (e.g. "2 eggs") → AI button estimates from description
- **AI button**: In g mode, calls `/estimate` (name + weight). In qty mode, calls `/describe` (description string)
- **Calorie auto-calc**: $P \times 4 + C \times 4 + F \times 9$ recalculated live as macros are typed
- **patch() function**: Central form updater — when weight changes in g mode and `basePerGram` is set, automatically rescales protein/carbs/fat proportionally
- **Validation** (`validateEntry()`): Checks calories match macros within $\pm 12\%$, total macros $\leq$ food weight $\times 1.1$, no single macro $> 95\%$ of weight

- MealChips selector: breakfast/pre_workout/lunch/post_workout/dinner/snack
- If editing an existing entry: deletes the old one then adds the new one

Suggest Mode

- Select **When** (Morning / Pre-Workout / Post-Workout / Rest Day / Evening)
- Select **Size** (Full Meal / Light)
- Calls `suggestMeal()` → hits AI Worker `/suggest`
- **Result view** shows:
 - Meal name + total macros (4-column grid)
 - Ingredient list with each item's name, amount, and P/C/F
 - "Log N items →" button: iterates ingredients array, adds each as a separate food log entry with the correct meal type (morning → breakfast, post_workout → post_workout, etc.)
 - "← Back" resets to the selector

Sub-components inside FoodScreen:

ComponentPurpose

|---|---|

`FoodCard`Individual food entry card with Edit/Remove/Info

`IngredientBreakdown`Shows per-ingredient name + amount + kcal + P/C/F

`MacroInp`Styled number input for protein/carbs/fat with color label

`MealChips`Scrollable row of meal type buttons `ErrMsgBox`Red error message box

`bigBtn()`Helper function returning a `CSSProperties` object for primary buttons

[ProfileSetupScreen.tsx](#) — Stats + Settings

Shows and edits the user's body profile. Three sections:

- **Stats grid** — weight, height, age, gender displayed as cards with colored top borders
- **Edit form** — inputs for each stat + activity level select

- **Account section** — "Change PIN", "Clear PIN", logout button

On save: calls `updateProfile()` to write to IndexedDB, then `setUser()` to update `authStore` (triggers macro recomputation in `useNutrition`).

8. Components

`TrainingPicker.tsx`

Four mode buttons: Rest, Strength, Cardio, Hybrid. Each has a sport-specific color:

- Rest → `#0288D1` (cyan)
- Strength → `#00A651` (green)
- Cardio → `#E65100` (orange)
- Hybrid → `#7B1FA2` (purple)

Active selection shows filled background, inactive shows subtle outline.

`onSelect(type)` callback triggers `logDay()` in `HomeScreen`.

`StravaCard.tsx`

Handles the full Strava integration UI:

States:

- Loading → "Loading..."
- Not connected → "Connect Strava" button that calls `getStravaAuthUrl()` to get the OAuth URL, then redirects
- Connected → athlete name, weekly km, run list, disconnect button

`RunRow` component:

Each Strava run shows: distance (km), name, date, pace (min/km), heart rate if available.

- **Log button (+/✓)** — calls `addRunKm()` to add to weekly load in `nutritionStore`. Shows checkmark when logged.
- **Share button** — opens a sub-panel with "Add photo" / "No photo →" options.

Calls `shareRunCard()` which renders a Canvas image (1080×1350) and shares via Web Share API (or downloads on desktop).

`WeatherBanner.tsx`

Displays current weather conditions with a color-coded alert level:

- Green (none) = good to train
- Yellow (caution) = hot/humid, adjust pace
- Red (danger) = extreme heat, stay indoor

Shows: temperature, city, humidity, UV index, alert message, suggested training swap if applicable, extra hydration recommendation.

`MacroBar.tsx`, `MacroCard.tsx`, `RingProgress.tsx`

Utility display components used across the app for macro progress visualization.

9. Stores (Zustand)

`authStore.ts`

```
interface AuthState {  
  
  user: BackendUser | null;      // loaded from IndexedDB on boot  
  
  pinVerified: boolean;          // stored in sessionStorage (clears on  
tab close)  
  
  setUser(u)                     // called after profile load/update  
  
  setPinVerified(v)              // called by PinScreen on success  
  
  logout()                       // clears session, resets nutrition store
```

```
| }
```

PIN session: Uses `sessionStorage` key `fuelsync_pin_verified`. This means PIN is required on fresh tab/window but not on page reload within the same session.

`nutritionStore.ts`

The central state store. Persisted to `localStorage` under key `fuelsync-nutrition-v2`.

What's persisted: `weeklyLoad` and `todayLog` only — everything else (targets, weather) is recomputed on mount.

```
interface NutritionState {

  todayLog: DailyLog | null          // today's training type selection

  targets: MacroTargets | null      // computed macros for today

  weeklyLoad: WeeklyLoad            // running totals for the week

  weather: WeatherConditions | null // current weather (not persisted)

  environmentAlert: EnvironmentAlert | null

  setTargets(targets)

  setTodayLog(log)

  setWeather(weather, alert)

  addRunKm(km, name, source)         // adds to loggedRuns + totalRunKm

  removeRunKm(km, name)              // finds first matching run, removes
  it
```

```

    renameRun(idx, newName) // renames run at index

    resetWeeklyRuns() // clears loggedRuns + totalRunKm

    logWorkoutComplete(km, sets) // calls
    nutritionEngine.updateWeeklyLoad()

    resetDay() // clears todayLog + targets

    resetAll() // full reset (called on logout)

}

```

10. Hooks

[useNutrition.ts](#)

The main hook that connects the UI to the nutrition engine. All screens use this.

Returned values:

ValueDescription

|---|---|

`profile` Converts `BackendUser` (authStore) to `UserProfile` (engine format). `null` if body stats incomplete. `todayLog` From nutritionStore `targets` From nutritionStore (computed macros) `weeklyLoad` From nutritionStore `weather` From nutritionStore `environmentAlert` From nutritionStore `logDay(type, time?)` Picks training type for today. Calls `computeMacros()` and saves targets. Checks leg fatigue gate first. `refreshWeather()` Gets GPS coordinates via `navigator.geolocation`, fetches from OpenWeatherMap, evaluates alert, saves to store. `getMacroBreakdown()` Returns full `MacroBreakdown` including carb timing window. `logWorkoutComplete(km, sets)` Updates weekly load after a workout. `resetDay()` Clears today's selection.

`toUserProfile(user)` — converts the flat `BackendUser` shape from `authStore` into the `UserProfile` shape the nutrition engine expects.

Auto-recompute: A `useEffect` watches for changes to the user's body stats (weight, height, age, etc.) and automatically recomputes macro targets when they change.

11. API Layer

`client.ts` — Worker HTTP Client

```
const AI_WORKER      = VITE_AI_WORKER_URL      // → fuelsync-  
ai.danielmarkman.workers.dev  
  
const STRAVA_WORKER = VITE_STRAVA_WORKER_URL // → fuelsync-  
strava.danielmarkman.workers.dev  
  
workerPost(worker, path, body): Promise  // POST with JSON body  
  
workerGet(worker, path): Promise  // GET request
```

Both functions throw `Error` if response is not ok, using the `error` field from the JSON response body.

`auth.ts` — Profile CRUD (IndexedDB)

```
getProfile(): Promise           // reads first row from db.profile  
  
createProfile(data): Promise    // clears + adds (single-profile  
design)  
  
updateProfile(data): Promise    // partial update
```

```
clearProfile(): Promise // called on logout/wipe
```

`BackendUser` is the public shape exposed to the app — it maps from `LocalProfile` (the raw IndexedDB interface).

`localFood.ts` — Food Log CRUD + AI

```
// Data types

interface FoodLog { id, food_name, calories, protein, carbs, fat,
weight_grams, meal_type, image_url, logged_at }

interface IngredientItem { name, amount, calories, protein, carbs, fat }

interface AIEstimate { food_name, estimated_weight_grams, calories,
protein, carbs, fat, confidence, breakdown?, imageUrl?, ingredients? }

// IndexedDB operations

getLogs(date: string): Promise // query by YYYY-MM-DD date index

addLog(entry): Promise // adds to db.food_logs

deleteLog(id: string): Promise // removes by numeric id

// AI Cloudflare Worker calls

estimateByWeight(name, grams): Promise // POST /estimate

estimateByDescription(desc): Promise // POST /describe

suggestMeal(context, size): Promise // POST /suggest

analyzeByImage(file): Promise // POST /analyze (base64 image)
```

`analyzeByImage` reads the File object using `FileReader`, converts to base64, then sends `{ base64, mimeType }` to the worker.

`strava.ts` — Strava Integration

```
getStravaAuthUrl(): Promise<{ url: string }> // GET /auth-url
from Strava Worker

connectStrava(tokens): Promise<{...}> // saves tokens to
IndexedDB profile

getStravaStats(): Promise // POST /stats with
access_token

disconnectStrava(): Promise<{ connected: false }> // clears tokens
from IndexedDB profile
```

Token refresh logic (`refreshIfNeeded()`):

- Reads tokens from IndexedDB profile
- Checks if `stravaExpiresAt` is > 60 seconds in the future
- If expired: calls Strava Worker `/refresh` endpoint with the refresh token
- Saves new tokens back to IndexedDB
- Returns the valid access token

12. Utilities

`sounds.ts` — Success Sound

```
playFoodLogSound(): void
```

Plays a **C5 → E5 → G5 ascending arpeggio** using the Web Audio API. Uses sine wave oscillators with a 0.02s attack and 0.32s decay. Staggered by 0.1s between

notes. Plays on successful food log.

No external audio files — generated entirely in the browser.

`runShareCard.ts` — Run Share Image

```
generateRunCard(run, photo?): HTMLCanvasElement // generates 1080×1350
canvas

shareRunCard(run, photoFile?): Promise // generates + shares/downloads
```

Canvas layout (1080×1350px):

- Top 58% (783px) — photo zone: cover-fits user's photo, or dark gradient with faint km watermark
- Bottom 42% (567px) — dark panel (#0F0C0A) with Strava orange stripe
- Panel contains: big distance number, KM label, stats row (pace / duration / elevation / heart rate), "VIA STRAVA" footer + date

Sharing: Uses `navigator.share({ files: [pngFile] })` (Web Share API) if supported. Falls back to creating an element and programmatically clicking it for download.

13. Business Logic — Nutrition Engine

File: `apps/mobile/src/services/nutritionEngine.ts`

This is the core science of the app. Pure functions, no side effects, no async.

`calcBMR(profile)` → **number**

Mifflin-St Jeor equation:

```
BMR = 10×weight + 6.25×height - 5×age + (5 if male, -161 if female)
```

`calcTDEE(profile)` → **number**

`BMR × activityMultiplier`. Multipliers:

- sedentary: 1.2
- light: 1.375
- moderate: 1.55
- very_active: 1.725
- extra_active: 1.9

`computeMacros(profile, log, weeklyLoad)` → **MacroBreakdown**

The main engine function. Returns personalized macro targets based on today's training type.

Caloric adjustment by training type:

TypeCal adjustmentMacro split (P/C/F)Rationale

|---|---|---|---|

rest-15% (×0.85)	30/20/50	High fat for fuel, glycogen depletion, fat adaptation
strength+5% (×1.05)	38/35/27	High protein for MPS, timed carbs for workout window
cardio+10% (×1.10)	25/55/20	High carbs to saturate glycogen, low fat for gastric emptying
hybrid+8% (×1.08)	32/43/25	Balanced, slight carb emphasis for dual demands

Recovery adjustment: If `weeklyLoad.recoveryScore < 40`, calories are further reduced by 5% to prioritize tissue repair.

`buildCarbTimingWindow(plannedWorkoutTime, totalCarbsG, type)`

If a workout time is set, calculates a ±2h carb concentration window. Cardio = 60% of daily carbs in window, other types = 50%.

`checkLegFatigueGate(load, log)` → **{ blocked, message? }**

If `legFatigueScore >= 70` and today is a cardio day, returns `blocked: true` with a warning message. HomeScreen shows a confirm dialog letting the user override.

`updateWeeklyLoad(current, log, { addKm, addSets })` → **WeeklyLoad**

Updates leg fatigue score and recovery score:

- Cardio adds up to 20 fatigue points (proportional to km)
- Strength adds 1.2 points per set
- Rest subtracts 15 points (natural decay)
- Recovery score: rest/low intensity +10, moderate -5, high -15

`calcRemaining(targets, consumed)` → **MacroTargets**

Simple subtraction, floors at 0.

`macroAdherenceScore(targets, consumed)` → **number (0–100)**

Weighted compliance score: protein 40%, carbs 30%, fat 15%, calories 15%.

14. Business Logic — Weather Service

File: `apps/mobile/src/services/weatherService.ts`

`fetchWeather(lat, lon, apiKey)` → **WeatherConditions**

Calls OpenWeatherMap `/data/2.5/weather` with metric units. Returns temp, humidity, UV index, description, city name.

`evaluateEnvironment(weather, plannedType)` → **EnvironmentAlert**

Thresholds:

- High temp: $\geq 30^{\circ}\text{C}$
- Danger temp: $\geq 36^{\circ}\text{C}$
- High humidity: $\geq 80\%$
- Danger humidity: $\geq 90\%$
- High UV: ≥ 8

Logic:

- All below threshold → `level: 'none'`, good to train
- Danger tier (extreme heat or humidity) → `level: 'danger'`, suggests swapping to indoor strength, +1000ml hydration
- Caution tier → `level: 'caution'`, lists what's elevated, +400–600ml hydration. If it's a cardio day and it's hot/humid, suggests swapping to upper-body strength.

`suggestWeeklyPivot(blockedDay, schedule)`

Given a blocked day (e.g. danger heat on a cardio day), finds the nearest rest or strength day in the next 3 days and swaps them. Used for weekly planning.

15. Security — PIN System

File: `apps/web/src/lib/pin.ts`

Hashing: PBKDF2

```
// 100,000 iterations of PBKDF2-SHA256

// Random 16-byte salt per PIN (stored as hex in IndexedDB)

async function pbkdf2Hash(text, salt): Promise
```

Uses the native **Web Crypto API** (`crypto.subtle`). No external libraries. The PIN is never stored in plain text anywhere.

Public functions

FunctionDescription

|---|---|

`setupPin(pin)`Generates salt, hashes PIN, clears `pin_state` table, adds new record
`setupSecurityQuestion(question, answer)`Hashes the answer (lowercased +

trimmed), stores with question `hasPin()` Returns true if `pin_state` table has any rows
`getSecurityQuestion()` Returns the security question string (not the answer)
`verifyPin(pin)` Hashes input + stored salt, compares. Tracks attempts, applies lockouts, wipes on limit
`verifySecurityAnswer(answer)` Same flow for security question answer
`resetPin(newPin)` Updates hash/salt, resets attempt counters
`clearPin()` Removes PIN completely (disables PIN lock)

Lockout Tiers

Total wrong attempts Lockout duration

|---|---|

330 seconds 65 minutes 1030 minutes **15Full database wipe** — `db.delete()` + `indexedDB.deleteDatabase('FuelSyncDB')`

Security question wrong answers: 5 attempts before database wipe.

16. Cloudflare Worker — AI (Gemini)

File: `workers/ai/index.ts` **Live URL:** `https://fuelsync-ai.danielmarkman.workers.dev` **Model:** `gemini-2.5-flash-lite` (free tier, higher RPM than 2.5-flash)

Environment variables (Wrangler secrets)

- `GEMINI_API_KEY` — Set via `wrangler secret put GEMINI_API_KEY`
- `ALLOWED_ORIGIN` — Set in `wrangler.toml` as `https://foodaniel.danielmms.site`

CORS

All responses include `Access-Control-Allow-Origin` set to the request's origin if it matches the app URL or starts with `http://localhost`. Otherwise defaults to the app URL. Handles `OPTIONS` preflight requests.

`gemini(apiKey, prompt, maxTokens, imageBase64?, imageMime?) → string`

Core function that calls the Gemini API. Builds the `contents[].parts` array:

- If image provided → adds { `inlineData: { mimeType, data: base64 }` } part first
- Adds text prompt part

`generationConfig: temperature: 0.1` (low for consistent nutritional facts),
`maxOutputTokens` as specified.

`parseJSON(text) → unknown`

Tries three strategies to extract JSON from Gemini's response:

- Direct `JSON.parse(text)`
- Extract from markdown code block (``json ...``)
- Regex match for first `{...}` object

JSON Schema sent to Gemini (MACRO_SCHEMA)

```
{  
  
  "food_name": "string",  
  
  "estimated_weight_grams": number,  
  
  "calories": number,  
  
  "protein": number,  
  
  "carbs": number,  
  
  "fat": number,  
  
  "confidence": "high|medium|low",
```

```

    "ingredients": [

      { "name": "string", "amount": "string", "calories": number, "protein":
number, "carbs": number, "fat": number }

    ]

  }

```

The `ingredients` array is **always required** — single-ingredient foods still get wrapped in the array (e.g. `[{ name: "banana", amount: "1 medium banana (118g)", ... }]`).

Endpoints

EndpointMethodInputDescription

|---|---|---|---|

`POST /estimate` `POST { food_name, weight_grams }` Estimates macros for a known food at a specific weight `POST /describe` `POST { description }` Analyses a free-text meal description ("2 eggs and toast") `POST /suggest` `POST { context, size }` Suggests a specific meal for an athlete ("Post-Workout", "big") `POST /analyze` `POST { base64, mimeType }` Identifies food in an image + estimates macros

Token budgets:

- `/estimate`: 768 tokens
- `/describe`, `/suggest`, `/analyze`: 1024 tokens (compound meals need more)

Deploy command

```
cd workers/ai
```

```
node_modules/.bin/wrangler deploy
```

(uses `wrangler@3` locally – Node 20 compatible)

17. Cloudflare Worker — Strava OAuth

File: `workers/strava/index.ts` **Live URL:** `https://fuelsync-strava.danielmarkman.workers.dev`

Environment variables

- `STRAVA_CLIENT_ID`, `STRAVA_CLIENT_SECRET` — Wrangler secrets
- `APP_URL` — `https://foodaniel.danielmms.site` (in `wrangler.toml`)
- `WORKER_URL` — `https://fuelsync-strava.danielmarkman.workers.dev` (in `wrangler.toml`)

Endpoints

`GET /auth-url`

Returns the Strava OAuth authorization URL with:

- `client_id` from env
- `redirect_uri` = `WORKER_URL/callback`
- Scopes: `read,activity:read_all`

The app redirects the user's browser to this URL.

`GET /callback`

Strava redirects here after the user approves access. The worker:

- Exchanges the `?code=` param for tokens via `POST`
`https://www.strava.com/oauth/token`
- Extracts athlete name + profile picture from the token response
- Redirects to `APP_URL?`
`strava_access_token=...&strava_refresh_token=...&strava_expires_at=...`
`&strava_athlete_name=...&strava_athlete_pic=...`

`App.tsx` reads these URL params and saves them to IndexedDB.

POST /refresh

Input: { refresh_token }

Exchanges a Strava refresh token for a new access token. Returns { access_token, refresh_token, expires_at }. Called automatically by strava.ts before any API request when the token has expired.

POST /stats

Input: { access_token }

Makes two parallel Strava API calls:

- GET /athlete — athlete name + id
- GET /athlete/activities?per_page=6 — last 6 activities

Then fetches GET /athletes/:id/stats for weekly/YTD/all-time totals.

Filters activities to type === 'Run' and computes:

- Distance in km (rounded to 2 decimal places)
- Duration as mm:ss
- Pace as m'ss" per km
- Heart rate average (if available)

Returns the full StravaStats shape used by StravaCard.tsx.

18. Infrastructure — Docker + Nginx

Dockerfile — Multi-stage Build

Stage 1 (builder): Node 20 Alpine

- Installs web app dependencies (npm ci in apps/web/)
- Copies source: apps/web/, apps/mobile/src/services/, shared/
- Runs npm run build → outputs to apps/web/dist/

Stage 2 (nginx): Alpine nginx

- Copies built static files to `/usr/share/nginx/html`
- Copies `nginx.conf`
- Exposes port 80

`nginx.conf`

```
location /api/      → proxy to foodaniel-backend:3000/ (legacy backend,
not used by PWA)

location /uploads/ → proxy to foodaniel-backend:3000/uploads/

location /          → try_files $uri $uri/ /index.html (SPA routing – any
path serves index.html)

location /assets/   → 1 year cache, immutable           (Vite content-
hashed JS/CSS)
```

Note: `location /` has `no-cache` so the browser always checks for a new `index.html` (which kicks off PWA service worker update check). Assets are aggressively cached because Vite names them with content hashes.

Deploy flow

```
cd /mnt/data/projects/mmswebsite

docker compose build foodaniel # runs Dockerfile

docker compose up -d foodaniel # restarts container with new image
```

Cloudflare Tunnel

Traffic from `foodaniel.danielmms.site` → Cloudflare edge → Cloudflare Tunnel daemon → Docker container on home server. The home server IP is never exposed.

19. Design System

File: apps/web/src/index.css

Color Palette (Israel flag inspired — white + blue)

```
--bg:      #EEF4FF / page background: light blue-white /

--surf:    #FFFFFF / cards: pure white /

--surf2:   #E4EEFF / input backgrounds, secondary surfaces /

--edge:    rgba(0,56,168,0.10) / card borders: subtle blue /

--text:    #0A1628 / primary text: near-black navy /

--muted:   #6878A0 / secondary text: muted blue-grey /

--blue:    #0038A8 / primary accent: Israel flag blue /

--blue2:   #1565E0 / gradient end: lighter blue /

--sky:     #2EA3F2 / sky blue /

--green:   #00A651 / protein, success /

--orange:  #E65100 / carbs, warning /

--purple:  #7B1FA2 / fat /
```

```
--red:    #C62828 / over-target, danger /  
  
--yellow: #F9A825 / caution /  
  
--cyan:   #0288D1 / rest day, fresh recovery /
```

CSS Classes

ClassEffect

|---|---|

`.nrc-pressscale(0.96)` on `:active` — tactile button press feel `.nrc-hero`Inter 900, -2px letter-spacing `.nrc-label`Inter 700, 9px, 3px tracking, uppercase `.nrc-aBase` for staggered entrance animations `.nrc-a1` through `.nrc-a6`Staggered slide-up animations (0.1s to 0.6s delays) `.volt-bar-fill`Blue progress bar fill

Typography

- Font: **Inter** (400–900 weights from Google Fonts)
- Heavy use of `fontWeight: 900` for numbers/heroes
- Negative letter-spacing (-1px to -4px) on large numbers for tight, athletic look
- Small labels: 9px, 700 weight, 2–4px letter-spacing, uppercase

20. Key Business Rules

Macro Validation (in `validateEntry()`)

When logging food manually:

- $P \times 4 + C \times 4 + F \times 9$ must be within **±12% of stated calories**
- $P + C + F$ must not exceed $\text{weight_grams} \times 1.1$ (10% buffer for water/fiber)
- No single macro $> \text{weight_grams} \times 0.95$ (e.g. you can't have 90g protein from 100g of food)

If any check fails, the form shows an error and the log is blocked.

Calorie Auto-Calculation

As the user types protein/carbs/fat, `calories` field auto-updates: `calories = P×4 + C×4 + F×9`. The user can override the calorie field manually, but a warning shows if it diverges >12% from the computed value.

Proportional Macro Scaling (g mode)

When the user changes the weight of a food (in grams mode) and macros were previously AI-estimated, the macros scale proportionally using `basePerGram` ratios stored from the AI estimate. This lets users say "I had 150g instead of 100g" and get correct macros automatically.

Carb Timing Window

Only computed when:

- Training type is NOT rest
- A workout time is set
- 50–60% of daily carbs (60% for cardio) concentrated ± 2 h around workout time

Leg Fatigue Gate

Only triggers for cardio days when `legFatigueScore >= 70`. The gate can be overridden by the user via a browser confirm dialog. Strava runs automatically add to the fatigue score via `addRunKm()` in the store.

Strava Token Auto-Refresh

Before every Strava API call, `refreshIfNeeded()` checks expiry. If expired, it calls the Strava Worker `/refresh` endpoint. The Worker holds the `client_secret` (which never touches the browser). New tokens are saved back to IndexedDB transparently.

Recovery Score

Increases on rest (+10) and low-intensity days, decreases on moderate (−5) and high (−15) intensity. When below 40, calories are reduced 5% by the macro engine to prioritize recovery over performance.

Generated from the full codebase on 2026-05-17.